

动态库接口函数使用说明

四川禾华盛电子科技有限公司

日期	版本	更改内容
2024. 4. 6	V1. 1	初始版本
2024. 10. 30	V1. 2	增加 CANFD 接口函数

目 录

1 简介.....	1
1.1 概述	1
1.2 运行环境	1
1.3 CAN, CANFD 启动及操作流程.....	1
2 结构体说明	2
2.1 Dev_Info	2
2.2 Can_Config	2
2.3 CanFD_Config	3
2.4 Can_Msg	4
2.5 CanFD_Msg	4
2.6 Can_Status	5
3 函数说明.....	8
3.1 Reg_HotPlug_Func	8
3.2 CAN_ScanDevice	8
3.3 CAN_GeDevType	8
3.4 CAN_GeDevPICK	9
3.5 CAN_OpenDevice	9
3.6 CAN_CloseDevice	9
3.7 CAN_ReadDevInfo	10
3.8 CAN_SetFilter	10
3.9 CAN_Reset	11
3.10 CAN_GetStatus	11
3.11 CAN_Init	12
3.12 CAN_Transmit	12
3.13 CAN_TransmitRt	13
3.14 CAN_GetReceiveNum	14
3.15 CAN_Receive	14
3.16 CANFD_Init	15
3.17 CANFD_Transmit	15
3.18 CANFD_TransmitRt	16
3.19 CANFD_GetReceiveNum	17
3.20 CANFD_Receive	17
4 注意事项	18

1 简介

1.1 概述

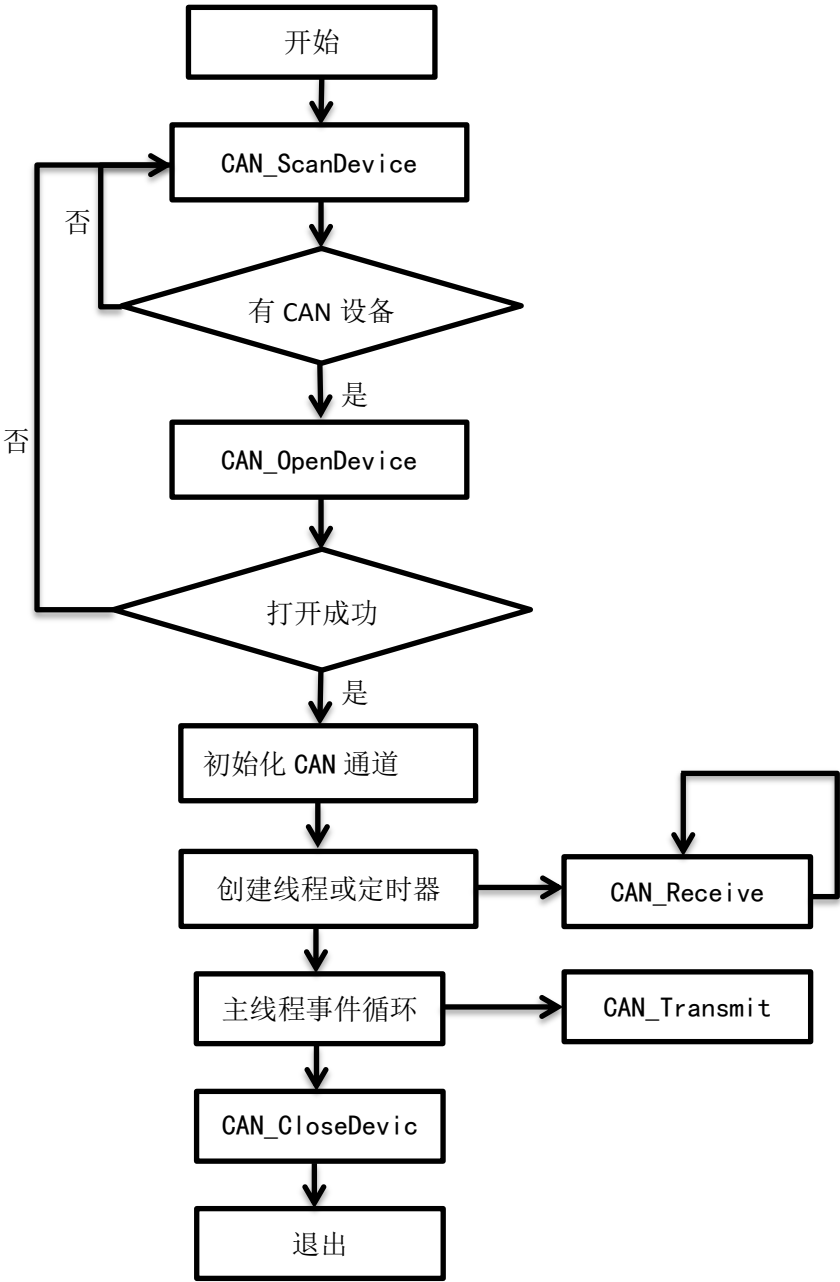
HCanbus.dll 配合 CAN 设备，可进行二次开发。

1.2 运行环境

支持 Windows 系统 (WinXP-WIN11), 32 位或 64 位, 支持 VC, VB, C#, Qt, delphi, labview, Matlab 等开发环境。

1.3 CAN, CANFD 启动及操作流程

CAN, CANFD 启动和操作流程图如下：



2 结构体说明

2.1 Dev_Info

Dev_Info 结构体定义了 CAN, CANFD 硬件设备信息。

```
typedef struct _Dev_Info {  
    char  HW_Type[32];  
    char  HW_Ser[32];  
    char  HW_Ver[32];  
    char  FW_Ver[32];  
    char  MF_Date[32];  
} Dev_Info, *PDev_Info; 成员
```

1. HW_Type 设备型号 字符串。
2. HW_Ser 设备序列号, 字符串, 每个设备序列号唯一, 可用于加密。
3. HW_Ver 硬件版本, 字符串。
4. FW_Ver 固件版本, 字符串。
5. MF_Date 生产日期, 字符串。

2.2 Can_Config

Can_Config 结构体定义了初始化 CAN 的配置。结构体将在 CAN_Init 函数中被填充, 即初始化之前, 要先填好这个结构体变量

```
typedef struct _Can_Config {  
    unsigned int    Baudrate;  
    unsigned short  Pres;  
    unsigned char   Tseg1;  
    unsigned char   Tseg2;  
    unsigned char   SJW;  
    unsigned char   Config;  
    unsigned char   Model;  
    unsigned char   Reserved;  
} Can_Config, *P_Can_Config;
```

成员

1. Baudrate 波特率, 比如设 500k, baudrate=500000
2. Pres 分频器
3. Config 配置信息: 0x01 接通内部电阻(仅对含有内部电子开关的设备有效), 0x02 离线唤醒, 0x04 自动重传, 可组合
4. SJW 同步跳转宽度
5. Model 工作模式: 0 正常模式, 1 环回模式, 2 静默模式, 3 静默环回模式。
6. Tseg1 时间段 1

7. Tseg2 时间段 2

8. Reserved 保留

说明: Pres, Tseg1, Tseg2 如果与 Baudrate 不匹配, 动态库会自动按 Baudrate 重新计算

Pres, Tseg1, Tseg2, SJW, 并且采样点设置为 75%左右。一般情况可以只设置波特率,

Pres, Tseg1, Tseg2 填 0。也可以借助波特率计算器详细配置这些参数。

2.3 CanFD_Config

CanFD_Config 结构体定义了初始化 CAN 的配置。结构体将在 CAN_Init 函数中被填充, 即初始化之前, 要先填好这个结构体变量

```
typedef struct _Can_Config {  
    unsigned int    NomBaud;  
    unsigned int    DatBaud;  
    unsigned short  NomPre;  
    unsigned char   NomTseg1;  
    unsigned char   NomTseg2;  
    unsigned char   NomSJW;  
    unsigned char   DatPre;  
    unsigned char   DatTseg1;  
    unsigned char   DatTseg2;  
    unsigned char   DatSJW;  
    unsigned char   Config;  
    unsigned char   Model;  
    unsigned char   Cantype;  
}CanFD_Config, *P_CanFD_Config;
```

成员

1. NomBaud 常规波特率, 比如设 500k, baudrate=500000。
2. DatBaud 数据波特率, 比如设 5M, baudrate=5000000。
3. NomPre 常规波特率分频器
4. NomTseg1 常规波特率时间段 1
5. NomTseg2 常规波特率时间段 2
6. NomSJW 常规波特率同步跳转宽度
7. DatPre 数据波特率分频器
8. DatTseg1 数据波特率时间段 1
9. DatTseg2 数据波特率时间段 2
10. DatSJW 数据波特率同步跳转宽度
11. Config 配置信息: 0x01 接通内部电阻(仅对含有内部电子开关的设备有效), 0x02 离线唤醒, 0x04 自动重传, 可组合。
12. Model 工作模式: 0 正常模式, 1 环回模式, 2 静默模式, 3 静默环回模式。

13. Cantype CAN 打开方式: 0 常规 CAN, 1 IOS CANFD, 2 Non-ISO CANFD

说明: 常规波特率、数据波特率的 Pres, Tseg1, Tseg2 如果与 Baudrate 不匹配, 动态库会自动按 Baudrate 重新计算 Pres, Tseg1, Tseg2, SWJ, 并且采样点设置为 75%左右。可以借助波特率计算器详细配置这些参数。

2.4 Can_Msg

Can_Msg 结构体表示帧的数据结构。在发送函数 Transmit 和接收函数 Receive 中被用来传送 CAN 信息帧:

```
typedef struct _Can_Msg {
    unsigned int    ID;
    unsigned int    TimeStamp;
    unsigned char   FrameType;
    unsigned char   DataLen;
    unsigned char   ExternFlag;
    unsigned char   RemoteFlag;
    unsigned char   BusSatus;
    unsigned char   ErrSatus;
    unsigned char   TECounter;
    unsigned char   RECounter;
    unsigned char   Data[8];
}Can_Msg, *P_Can_Msg;
```

成员:

1. ID 报文帧 ID。
2. TimeStamp 帧数据的时间标识, 单位微秒。
3. FrameType 帧类型。定义见表 1。
4. DataLen 数据长度, 最大值是 8。
5. ExternFlag 是否是扩展帧。0:标准帧 (11 位帧 ID), 1:扩展帧 (29 位帧 ID)。
6. RemoteFlag 是否是远程帧。0:数据帧, 1:远程帧。
7. BusSatus 总线状态, 定义见表 2。
8. ErrSatus 错误状态, 定义见表 3。
9. TECounter 发送错误计数。
10. RECounter 接收错误计数。
11. Data CAN 报文的数据。有效字节为 DataLen。

2.5 CanFD_Msg

Can_Msg 结构体表示帧的数据结构。在发送函数 Transmit 和接收函数 Receive 中被用来传送 CAN 信息帧:

```
typedef struct _Can_Msg {
```

```
    unsigned int    ID;
    unsigned int    TimeStamp;
    unsigned char    FrameType;
    unsigned char    DLC;
    unsigned char    ExternFlag;
    unsigned char    RemoteFlag;
    unsigned char    BusSatus;
    unsigned char    ErrSatus;
    unsigned char    TECounter;
    unsigned char    RECounter;
    unsigned char    Data[64];
}CanFD_Msg, *P_CanFD_Msg;
```

成员:

1. ID 报文帧 ID。
2. TimeStamp 帧数据的时间标识, 单位微秒。
3. FrameType 帧类型。定义见表 1。
4. 数据 DLC, 不是数据长度, 最大值是 15, DLC 与数据长度关系见表 4。
5. ExternFlag 是否是扩展帧。0:标准帧 (11 位帧 ID), 1:扩展帧 (29 位帧 ID)。
6. RemoteFlag 是否是远程帧。0:数据帧, 1:远程帧。(CANFD 取消了远程帧)
7. BusSatus 总线状态, 定义见表 2。
8. ErrSatus 错误状态, 定义见表 3。
9. TECounter 发送错误计数。
10. RECounter 接收错误计数。
11. Data CAN 报文的数据。有效字节为 DLC 对应的数据长度。

2.6 Can_Status

Can_Status 结构体用于获取 CAN 设备状态。

```
typedef struct _Can_Status {
    unsigned char    BusSatus;
    unsigned char    ErrSatus;
    unsigned char    TECounter;
    unsigned char    RECounter;
    unsigned int    TimeStamp;
}Can_Status, *P_Can_Status;
```

成员

1. BusSatus 总线状态, 定义见表 2。
2. ErrSatus 错误状态, 定义见表 3。
3. TECounter 发送错误计数。

- 4. RECounter 接收错误计数。
- 5. TimeStamp 产生状态时的时间戳

表 1 FrameType 定义 (组合)

位	值	定义描述
1	0	接收帧
	1	发送帧 (0x01) ①
2	0	传输成功
	1	传输失败 (0x02)
3	0	常规 CAN
	1	CANFD (0x04)
4	1	FDBRS (0x08) ②
5	1	定时发送开始 (0x10)
6	1	定时发送结束 (0x20)
7	1	发送数据回传 echo (0x40) ③

注：①发送数据不必添加此标识；接收的数据有发送帧，表示发送数据的回传；
②发送 FDBRS 数据，CANFD 需置位 (FrameType = 0x0c)
③发送数据添加回传 (0x40)，发送完成后会将此数据回传到接收队列。

表 2 CAN 总线状态 (组合)

位	值	状态描述
1	1	错误警告 (0x01)
2	1	错误被动 (0x02)
3	1	CAN 总线离线 (0x04)
4	1	总线正常 (0x08)
5	1	发送邮箱总裁错误 (0x10)
6	1	发送邮箱空 (0x20) ④
7	1	接收寄存器溢出 (0x40)
8	1	接收寄存器满 (0x80)

注：④发送邮箱空，表示发送邮箱没有数据等待发送，可以发送数据。

表 3 CAN 错误状态(非组合)

序号	值	状态描述
1	0x01	位填充错
2	0x02	格式(Form)错
3	0x03	确认(ACK)错
4	0x04	隐形位错
5	0x05	显性位错
6	0x06	CRC 错

表 4 DLC 与数据长度对于关系

序号	DLC	数据长度
1	0	0
2	1	1
3	2	2
4	3	3
5	4	4
6	5	5
7	6	6
8	7	7
9	8	8
10	9	12
11	10	16
12	11	20
13	12	24
14	13	32
15	14	48
16	15	64

3 函数说明

3.1 Reg_HotPlug_Func

描述

注册 CAN, CANFD 设备热拔插函数, 可用于检测提醒插入拔出 CAN, CANFD 设备。

```
int __stdcall Reg_HotPlug_Func(void(*pfunc)(void));
```

参数

pfunc 热拔插函数指针

返回值

0 表示成功, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
int ret;
CanbustestDlg dlg;
m_pMainWnd = &dlg;
pdig = &dlg;
Reg_HotPlug_Func(&func_hotplugs);
```

3.2 CAN_ScanDevice

描述

此函数用于扫描 CAN, CANFD 设备。

```
int __stdcall CAN_ScanDevice(void);
```

参数

无

返回值

返回扫描到 CAN 设备的个数, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
int devs = 0;
devs = CAN_ScanDevice();
```

3.3 CAN_GeDevType

描述

此函数用于获取 CAN, CANFD 设备类型。

```
int __stdcall CAN_GeDevType ((unsigned int devNum);
```

参数

devNum 设备通道号

返回值

0: 常规 CAN, 1: CANFD。

示例:

```
#include "HCanbus.h"
int ret = CAN_GeDevType (0);
if(ret == 1) {cout<<"设备是 CANFD! ";}
else {cout<<"设备是 CAN2.0! ";}
```

3.4 CAN_GeDevPICK

描述

此函数用于获取 CAN, CANFD 设备频率, 可用于计算波特率。

```
int __stdcall CAN_GeDevPICK ((unsigned int devNum);
```

参数

devNum 设备通道号

返回值

返回 can 设备频率。

示例:

```
#include "HCanbus.h"
int ret = CAN_GeDevType (0);
```

3.5 CAN_OpenDevice

描述

此函数用于打开 CAN,CANFD 设备。

```
int __stdcall CAN_OpenDevice(unsigned int devNum);
```

参数

devNum 设备通道号

返回值

0 表示操作成功, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
int ret = CAN_OpenDevice(0);
if(ret != 0) {cout<<"打开 CAN 设备失败! "; return;}
```

3.6 CAN_CloseDevice

描述

此函数用于关闭 CAN,CANFD 设备。

```
int __stdcall CAN_CloseDevice(unsigned int devNum);
```

参数

devNum 设备通道号

返回值

0 表示操作成功, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
int ret = CAN_CloseDevice (0);
if(ret != 0) {cout<<"关闭 CAN 设备失败! "; return;}
```

3.7 CAN_ReadDevInfo

描述

此函数用于读取 CAN,CANFD 设备信息。

```
int __stdcall CAN_ReadDevInfo(unsigned int devNum, PDev_Info devinfo);
```

参数

devNum 设备通道号

devinfo 设备信息结构体

返回值

0 表示操作成功, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
Dev_Info devinfo;
int ret = CAN_ReadDevInfo (0, &devinfo);
if(ret != 0)
{
    cout<<"设备型号:"<< devinfo. HW_Type;
    cout<<"序列号:"<< devinfo. HW_Ser;
    cout<<"硬件版本:"<< devinfo. HW_Ver;
    cout<<"固件版本:"<< devinfo. FW_Ver;
    cout<<"生产日期:"<< devinfo. MF_Date;
    return;
}
```

3.8 CAN_SetFilter

描述

此函数用于设置 CAN,CANFD 过滤器。

```
int __stdcall CAN_SetFilter(unsigned int devNum, char number, char type,
unsigned int ftID, unsigned int ftMask, char enable);
```

参数

devNum 设备通道号

number 过滤器编号, 范围 0-13

type 过滤器类型, 0: 2 个 16 位过滤器, 1: 一个 32 位过滤器。

ftID 标识符

ftMask 屏蔽位

enable 是否使能, 0: 非使能, 1: 使能

返回值

0 表示操作成功, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
```

```
int ret;  
ret = CAN_SetFilter(0,0,1,0,0,1);  
if(ret != 0) {cout<<"操作失败!"; return;}
```

注：调用 CAN_Init 时，如果没有提前设置过滤器，会自动启用一个过滤器 0，ftID 和 ftMask 均为 0 并使能，接收全部数据。可调用 CAN_SetFilter，改变或关闭此过滤器。

3.9 CAN_Reset

描述

此函数用于复位 CAN, CANFD 过设备。

```
int __stdcall CAN_Reset(unsigned int devNum);
```

参数

devNum 设备通道号

返回值

0 表示操作成功，-1 表示操作失败。

示例：

```
#include "HCanbus.h"  
int ret;  
ret = CAN_Reset (0);  
if(ret != 0) {cout<<"操作失败!"; return;}
```

注意：如果之前有设置过滤器，调用 CAN_Reset 后，需要再设置一次。

3.10 CAN_GetStatus

描述

此函数用于获取 CAN, CANFD 设备状态，包括总线状态，错误信息。

```
int __stdcall CAN_GetStatus(unsigned int devNum, P_Can_Status status);
```

参数

devNum 设备通道号

status CAN 设备状态首指针。

返回值

0 表示操作成功，-1 表示操作失败。

示例：

```
#include "HCanbus.h"  
int ret;  
Can_Status status;  
ret = CAN_Receive (0, &status);
```

3.11 CAN_Init

描述

此函数用于初始化 CAN 设备。

```
int __stdcall CAN_Init(unsigned int devNum, P_Can_Config pInitConfig);
```

参数

devNum 设备通道号

返回值

0 表示操作成功, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
int ret;
Can_Config Cancfg;
cancfg.Model = 0; //正常工作模式
baudrate = 1000*1000; //波特率为 1m
cancfg.Config = 0x01; //接通内部电阻 (仅对电子开关控制终端电阻设备有效)
cancfg.Config |= 0x02; //总线离线唤醒
cancfg.Config |= 0x04; //自动重发
ret = CAN_Init(0, &cancfg);
if(ret != 0) {cout<<"初始化 CAN 设备失败! "; return;}
cout <<"实际波特率"<< cancfg.Baudrate;
```

注意: 初始化成功后, pInitConfig 结构体的各成员被赋实际值, Baudrate 可用作评估误码率, 误差的参考; Pres, Tseg1, Tseg2 可计算采样点。

3.12 CAN_Transmit

描述

此函数用于发送 CAN 数据。

```
int __stdcall CAN_Transmit(unsigned int devNum, P_Can_Msg canmsg,
unsigned int items, int timeout);
```

参数

devNum 设备通道号

canmsg 要发送的数据帧数组的首指针, 数组实际大小不能小于 items。

items 要发送的帧数, 最大值为 100。

timeout 等待超时时间, 单位毫秒。小于 0: 长时间等待。为了提高发送成功率, 时间最好设置到 100 毫秒以上。

返回值

返回成功发送的帧数, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
int ret;
```

```

Can_Msg txmsg[100];           //结构体数组大小不能小于发送帧数
Txmsg[0].ID = 0x123;
txmsg[0].RemoteFlag = 0; //数据帧
txmsg[0].ExternFlag = 0; //标准帧
txmsg[0].DataLen = 8;
txmsg[0].Data[0] = 10; //给数据赋值
txmsg[0].Data[1] = 20;
ret = CAN_Transmit(0, txmsg, 1, 200);

```

3.13 CAN_TransmitRt

描述

此函数用于微秒级精密定时发送 CAN 数据。

```

int __stdcall CAN_TransmitRt(unsigned int devNum, P_Can_Msg canmsg,
    unsigned int items, unsigned int *txitems, int timeout);

```

参数

devNum 设备通道号

canmsg 要发送的数据帧数组的首指针。数组实际大小不能小于 items。

items 要发送的帧数，10 毫秒内发送的报文数，最小值 1，最大值 50。

Txitems 实际发送帧数，可以不关心。

timeout 等待超时时间，单位毫秒。小于 0: 长时间等待。为了提高发送成功率，时间最好设置到 100 毫秒以上。

返回值

返回成功发送的帧数，-1 表示操作失败。

示例：发送 1000 条间隔 1 毫秒的数据

```

#include "HCanbus.h"
int i, txitm;
Can_Msg txmsg[1000];           //结构体数组大小不能小于总帧数
for(i = 0; i < 1000; i++) //结构体数值赋值
{
    Txmsg[i].ID = 0x123;
    txmsg[i].RemoteFlag = 0; //数据帧
    txmsg[i].ExternFlag = 0; //标准帧
    txmsg[i].DataLen = 8;
    txmsg[i].FrameType = 0;
    txmsg[i].TimeStamp = i*1000 //设置时间戳，间隔 1 毫秒
    txmsg[i].Data[0] = 10; //给数据赋值
    txmsg[i].Data[1] = 20;
}
txmsg[0].FrameType |= 0x10; //定时发送开始标识
txmsg[999].FrameType |= 0x20; //定时发送结束标识
for(i = 0; i < 1000; i += 100) //10 毫秒内发送的报文数为 10

```

```
{  
    //所以设置每次发送 10 帧  
    CAN_TransmitRt (0,txmsg+i,10, &txitm,500);  
}
```

注意: 必须设置时间戳, FrameType|=0x10, 才能启动定时发送, 否则以普通方式发送。
设置最后一帧 FrameType|=0x20 才能结束定时发送。

3.14 CAN_GetReceiveNum

描述

此函数用以获取接收缓冲区中接收到但尚未被读取的帧数量。

```
int __stdcall CAN_GetReceiveNum(unsigned int devNum);
```

参数

devNum 设备通道号

返回值

返回 can 帧数量, -1 表示操作失败。

示例:

```
#include "HCanbus.h"  
int ret;  
ret = CAN_GetReceiveNum (0);  
if(ret == -1) {cout<<"操作失败! "; return;}  
cout <<"缓存 CAN 帧数"<< ret;
```

注意: 正常情况不必调用此函数, 直接调用 CAN_Receive 接收数据效率更高。

3.15 CAN_Receive

描述

此函数用于接收 CAN 数据。

```
int __stdcall CAN_Receive(unsigned int devNum, P_Can_Msg canmsg, int len,  
int timeout);
```

参数

devNum 设备通道号

canmsg 要接收的数据帧数组的首指针。数组实际大小不能小于 len。

len 预期接收的最多数据帧数, 最大值 2500。

timeout 等待超时时间, 单位毫秒。小于 0: 长时间等待。为减轻系统负担, 提高稳定性, 时间最好设置到 100 毫秒以上。

返回值

返回实际接收帧数, -1 表示操作失败。

示例:

```
#include "HCanbus.h"  
int ret;  
Can_Msg txmsg[1000]; //结构体数组大小不能小于接收帧数  
ret = CAN_Receive (0, txmsg, 1000, 500);
```



```
//一次最多能接收 1000 帧。500 毫内收到数据立即返回，超时返回 0。  
cout <<"接收帧数："<< ret;
```

3.16 CANFD_Init

描述

此函数用于初始化 CANFD 设备。

```
int __stdcall CANFD_Init(unsigned int devNum, P_CanFD_Config pInitConfig);
```

参数

devNum 设备通道号

返回值

0 表示操作成功，-1 表示操作失败。

示例：

```
#include "HCanbus.h"  
int ret;  
CanFD_Config Cancfg;  
cancfg.Model = 0; //正常工作模式  
cancfg.Cantype = 1; //CAN 模式：0:CAN, 1:IOS CANFD, 2:Non-ISO CANFD  
NomBaud = 1000000; //常规波特率为 1m  
DatBaud = 5000000; //数据波特率为 5m  
cancfg.Config = 0x01; //接通内部电阻(仅对电子开关终端电阻有效)  
cancfg.Config |= 0x02; //总线离线唤醒  
cancfg.Config |= 0x04; //自动重发  
ret = CANFD_Init(0, &cancfg);  
if(ret != 0) {cout<<"初始化 CAN 设备失败！"; return;}  
cout <<"实际波特率"<< cancfg.Baudrate;
```

注意：初始化成功后，pInitConfig 结构体的各成员被赋实际值，NomBaud、DatBaud 可用作评估误码率，误差的参考，NomPres, NomTseg1, NomTseg2 可计算常规采样点；DatPres, DatTseg1, DatTseg2 可计算数据采样点。

3.17 CANFD_Transmit

描述

此函数用于发送 CAN 数据。

```
int __stdcall CANFD_Transmit(unsigned int devNum, P_CanFD_Msg canmsg,  
unsigned int items, int timeout);
```

参数

devNum 设备通道号

canmsg 要发送的数据帧数组的首指针，数组实际大小不能小于 items。

items 要发送的帧数，最大值为 100。

timeout 等待超时时间, 单位毫秒。小于 0: 长时间等待。为了提高发送成功率, 时间最好设置到 100 毫秒以上。

返回值

返回成功发送的帧数, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
int ret;
CanFD_Msg txmsg[100];          //结构体数组大小不能小于发送帧数
Txmsg[0].ID = 0x123;
txmsg[0].FrameType = 0x0c;    //CANFD 数据, 以可变波特率方式加速发送
txmsg[0].DLC = 15;            //DLC 15 对应数据长度 64
txmsg[0].RemoteFlag = 0;    //数据帧
txmsg[0].ExternFlag = 0;    //标准帧
txmsg[0].Data[0] = 10;      //给数据赋值
txmsg[0].Data[1] = 20;
ret = CANFD_Transmit(0, txmsg, 1, 200);
```

3.18 CANFD_TransmitRt

描述

此函数用于微秒级精密定时发送 CAN 数据。

```
int __stdcall CANFD_TransmitRt(unsigned int devNum, P_CanFD_Msg canmsg,
unsigned int items, unsigned int *txitems, int timeout);
```

参数

devNum 设备通道号

canmsg 要发送的数据帧数组的首指针。数组实际大小不能小于 items。

items 要发送的帧数, 10 毫秒内发送的报文数, 最小值 1, 最大值 50。

Txitems 实际发送帧数, 可以不关心。

timeout 等待超时时间, 单位毫秒。小于 0: 长时间等待。为了提高发送成功率, 时间最好设置到 100 毫秒以上。

返回值

返回成功发送的帧数, -1 表示操作失败。

示例: 发送 1000 条间隔 1 毫秒的数据

```
#include "HCanbus.h"
int i, txitm;
CanFD_Msg txmsg[1000];          //结构体数组大小不能小于总帧数
for(i = 0; i < 1000; i++)      //结构体数值赋值
{
    Txmsg[i].ID = 0x123;
    txmsg[i].FrameType = 0x0c;    //CANFD 数据, 以可变波特率方式加速发送
    txmsg[i].DLC = 15;            //DLC 15 对应数据长度 64
```

```

    txmsg[i].RemoteFlag = 0; //数据帧
    txmsg[i].ExternFlag = 0; //标准帧
    txmsg[i].TimeStamp = i*1000 //设置时间戳，间隔 1 毫秒
    txmsg[i].Data[0] = 10;      //给数据赋值
    txmsg[i].Data[1] = 20;
}
txmsg[0].FrameType |= 0x10; //定时发送开始标识
txmsg[999].FrameType |= 0x20; //定时发送结束标识
for(i = 0; i < 1000; i += 100) //10 毫秒内发送的报文数为 10
{
    //所以设置每次发送 10 帧
    CANFD_TransmitRt (0, txmsg+i, 10, &txitm, 500);
}

```

注意: 必须设置时间戳, FrameType|=0x10, 才能启动定时发送, 否则以普通方式发送。
设置最后一帧 FrameType|=0x20 才能结束定时发送。

3.19 CANFD_GetReceiveNum

描述

此函数用以获取接收缓冲区中接收到但尚未被读取的帧数量。

```
int __stdcall CANFD_GetReceiveNum(unsigned int devNum);
```

参数

devNum 设备通道号

返回值

返回 can 和 canfd 帧数量, -1 表示操作失败。

示例:

```

#include "HCanbus.h"
int ret;
ret = CANFD_GetReceiveNum (0);
if(ret == -1) {cout<<"操作失败! "; return;}
cout <<"缓存帧数"<< ret;

```

注意: 正常情况不必调用此函数, 直接调用 CANFD_Receive 接收数据效率更高。

3.20 CANFD_Receive

描述

此函数用于接收 CAN 和 CANFD 数据。

```
int __stdcall CANFD_Receive(unsigned int devNum, P_CanFD_Msg canmsg, int
len, int timeout);
```

参数

devNum 设备通道号

canmsg 要接收的数据帧数组的首指针。数组实际大小不能小于 len。

len 预期接收的最多数据帧数, 最大值 2500。

timeout 等待超时时间, 单位毫秒。小于 0: 长时间等待。为减轻系统负担, 提高稳定性, 时间最好设置到 100 毫秒以上。

返回值

返回实际接收帧数, -1 表示操作失败。

示例:

```
#include "HCanbus.h"
int ret;
CanFD_Msg txmsg[1000]; //结构体数组大小不能小于接收帧数
ret = CANFD_Receive (0,txmsg,1000, 500);
//一次最多能接收 1000 帧。500 毫内收到数据立即返回, 超时返回 0。
cout <<"接收帧数: "<< ret;
```

4 注意事项

1. 按流程图启动 CAN 设备: 扫描设备-打开设备-初始化 CAN-收发数据, 关闭软件前请先关闭通道。
2. 为提高数据接收效率, 尽量创建线程接收数据。开多少个通道就创建多少个线程, 初始化 CAN 后让线程一直运行 CAN_Receive, 不要 sleep。无数据时 CAN_Receive 会让线程挂起等待直到超时, 不占 CPU 资源; 有数据后会立即返回 (不等接满预期的数据个数)。
3. 函数 CAN_Transmit 和 CAN_Receive 的参数 timeout, 是超时等待的时间, 不是执行一次函数需要消耗的时间。
4. CANFD 与 CAN 共用第 3.0-第 3.10 节函数。CANFD 设备可调用 CAN 初始化和收发函数, CAN 设备不能调用 CANFD 初始化和收发函数。
5. CANFD 需要初始化为 CANFD 设备才能接收 CANFD 数据, 赋值 CanFD_Msg 的成员 FrameType 为 04 或 0C 才能发送 CANFD 数据。
6. 动态库分 X86 和 X64 两个版本, 应该按开发环境的编译器版本位数选择对应的动态库, 而不是按操作系统的 32 位或 64 位。选择错误无法编译。